Last Updated: 7th October 2025

# Guide to Installing SFML 2.6.2 for use with Visual Studio Code (Windows & Mac Intel* & Apple Silicon)

Guide Prepared by: Liam Persad

## Disclaimer:

This guide is for general guidance only. The author has made every effort to provide accurate instructions and avoid errors, but it may not cover every scenario and does not guarantee a successful installation or error-free experience. Unexpected issues may occur depending on your system or setup.

If you would like to report any issues with this guide or any errors you encounter, please visit: https://github.com/liam2503/liam2503.github.io/issues and post your issue there. Your feedback will be very useful for the continued development of this guide.

Thank you to everyone who assisted in the testing and development of this guide.

** Due to differences between Intel and Apple Silicon versions of Mac OS, some tutorial images may appear differently, or files may be located in different locations. Please be sure to take note of any specific instructions for your architecture of Mac OS.*

# Contents:

# Section 1: Preparing Visual Studio Code for C++ Development

### 1. Installing Visual Studio Code (Windows & Mac)

Download and install the latest version of Visual Studio Code from https://code.visualstudio.com.
(At the time of writing, the latest version available is Version 1.104.2)

### 2. Installing a C++ Compiler

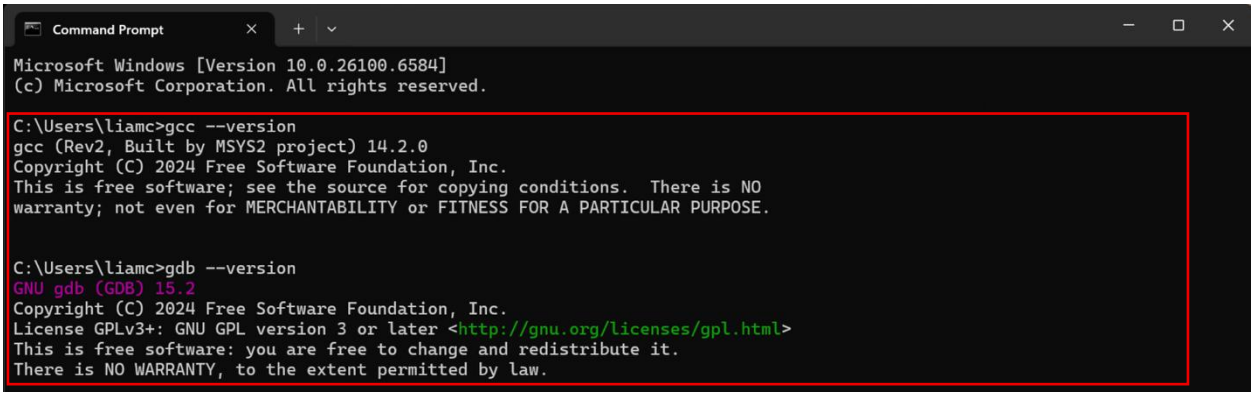
Follow the instructions for your respective operating system.
**(Mac Instructions begin on Page 6)**

## Windows:

To check if your C++ compiler installation is valid:

1. Open 'Command Prompt' from the Start Menu.
2. Verify the version of your compiler using `gcc --version`
3. Verify the version of your debugger using `gdb --version`

You should see a similar output below:

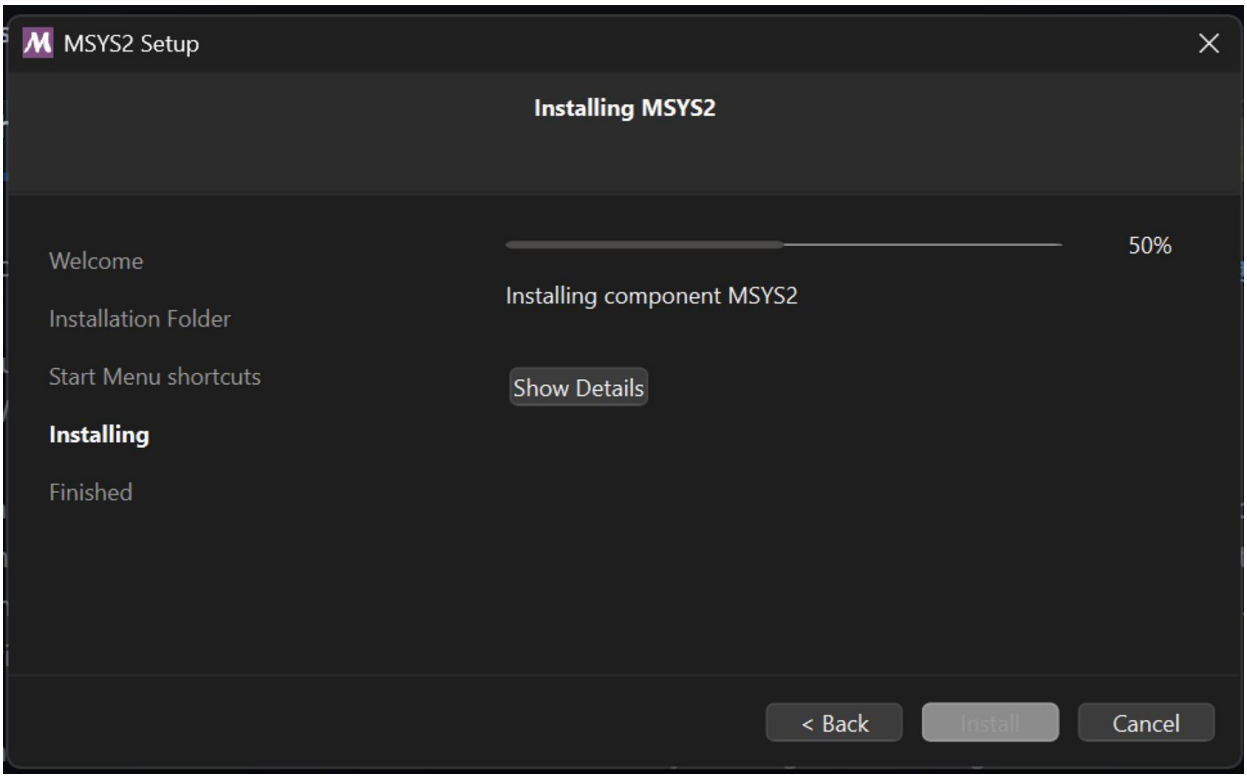



**If you cannot execute the above commands successfully, please install a C++ compiler as instructed by this guide.**

Microsoft recommends installing MinGW using ‘msys2’, which can be downloaded using this link:

https://github.com/msys2/msys2-installer/releases/download/2024-12-08/msys2-x86_64-20241208.exe

Once downloaded, complete the installation using the installation wizard.




After installation, the msys2 terminal will open.

Copy, paste and execute the following command into the terminal:
`pacman -S mingw-w64-x86_64-gcc`

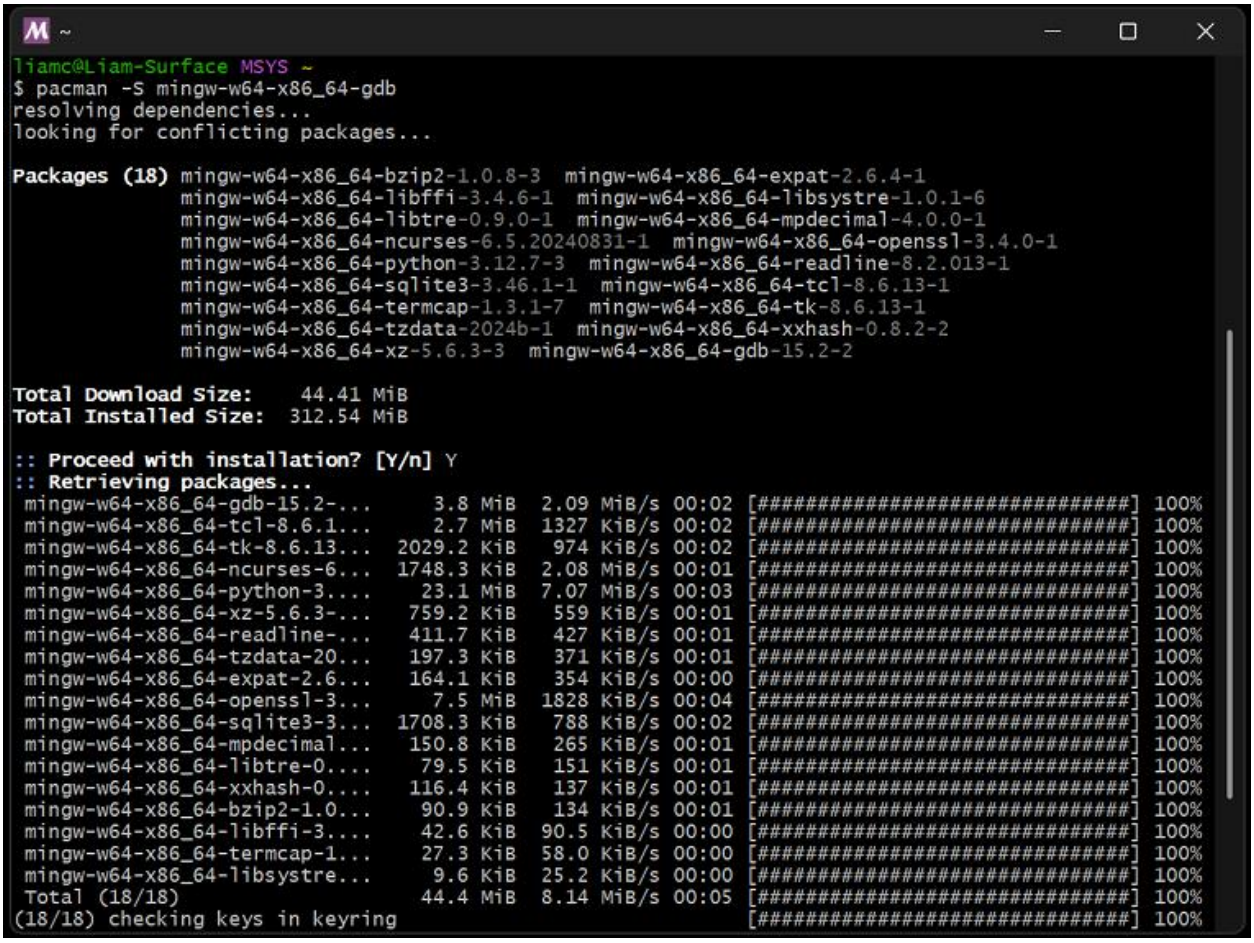
When prompted, enter ‘Y’ to accept the installation.

Once complete, do the same with the following command

`pacman -S mingw-w64-x86_64-gdb`

When prompted, enter ‘Y’ to accept the installation.

You should see a similar output below:

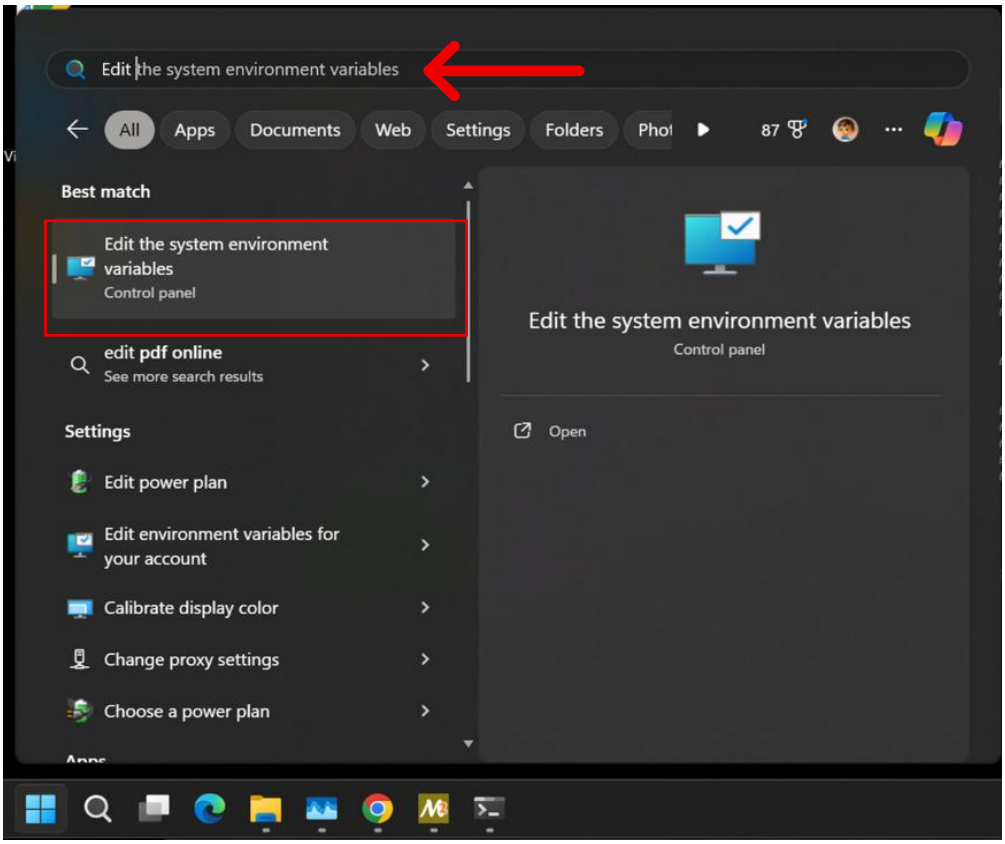



Once the installation is complete, open the ‘Start Menu’ and begin typing: `Edit the system environment variables` and select the highlighted option.

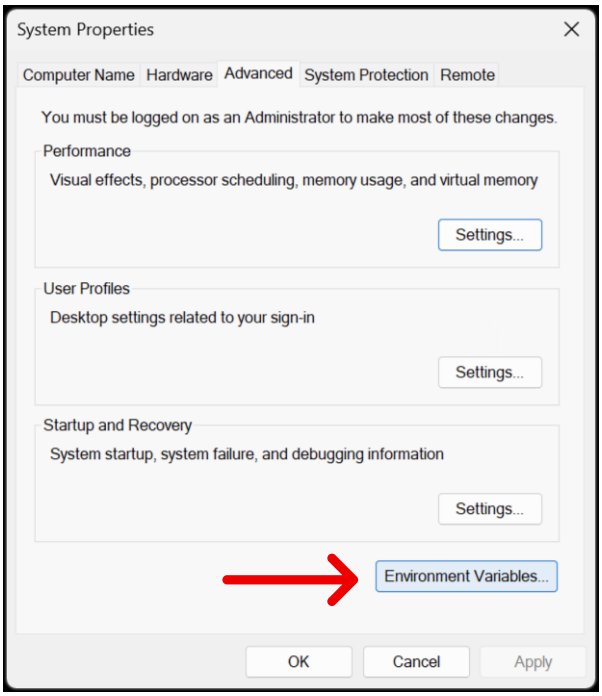
If this option doesn’t appear in the Start Menu (sometimes due to language settings), you can navigate the same menu as this:

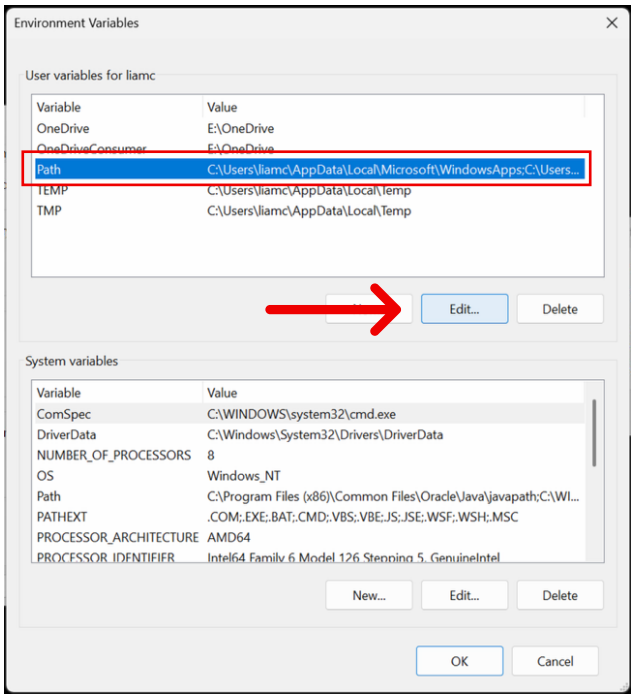
Right-click Start button and select ‘System’ (システム / 系统).

In the ‘System’ window, on the right side (or bottom), click ‘Advanced system settings’ (システムの詳細設定 / 高级系统设置).

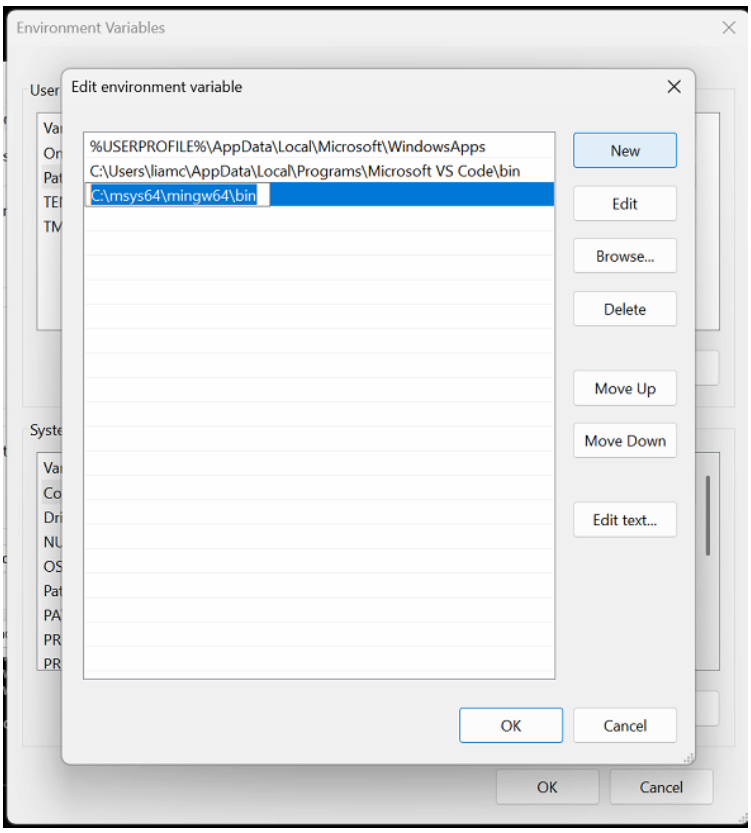



In the ‘System Properties’ menu that opens, select ‘Environment Variables’. Then under ‘User variables’, select ‘Path’ and click on the “Edit...” button.



Select the ‘New’ button and enter this path: `C:\msys64\mingw64\bin`

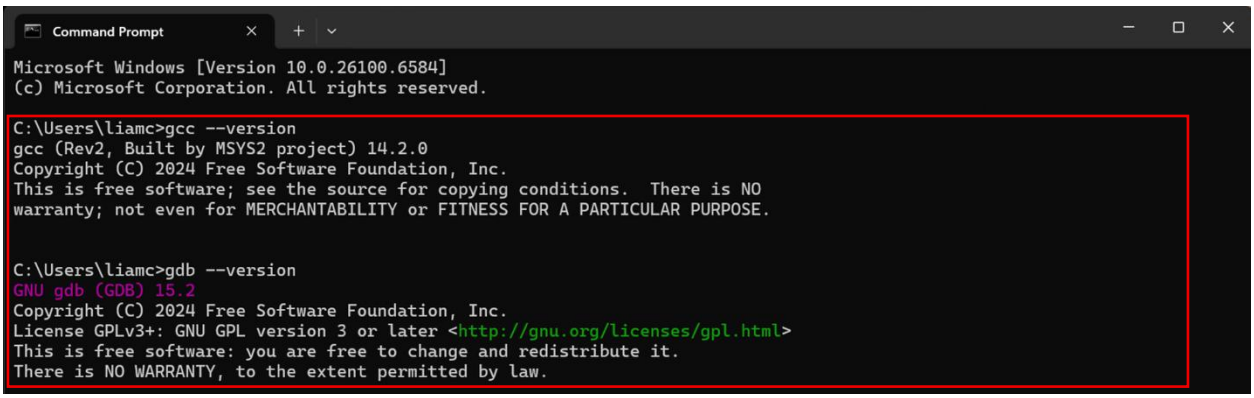



Then select ‘OK’ to save the path variable.

Once saved, verify your installation. To do this:

4. Open ‘Command Prompt’ from the Start Menu.
5. Verify the version of your compiler using `gcc --version`
6. Verify the version of your debugger using `gdb --version`

You should see a similar output below:

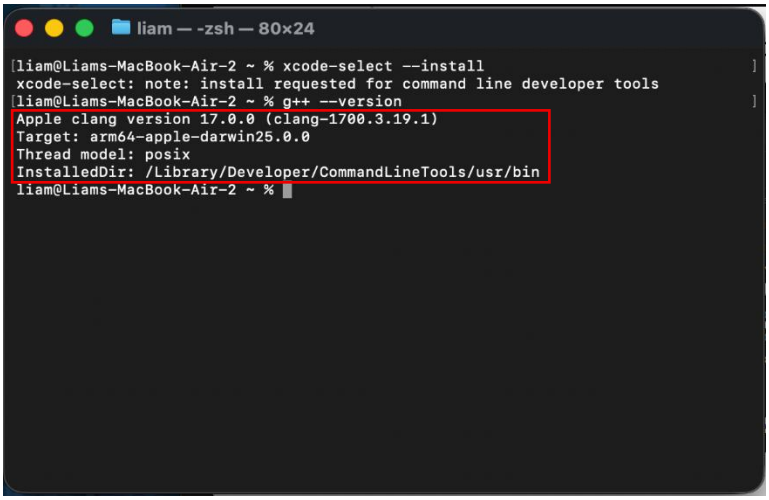


## Mac:

To check if your C++ compiler installation is valid:

1. Open ‘Terminal’.
2. Verify the version of your compiler using `g++ --version`

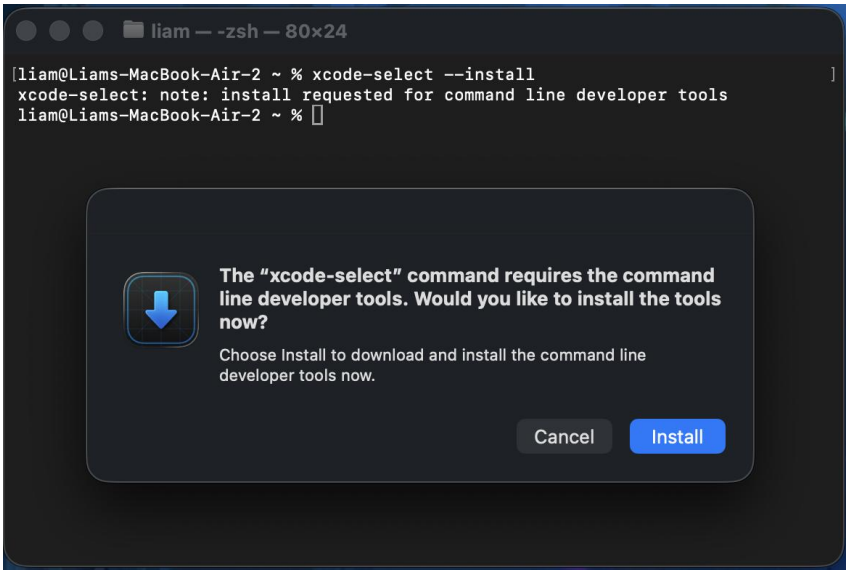
You should see a similar output below:




**If you cannot execute the above commands successfully, please install a C++ compiler as instructed by this guide.**

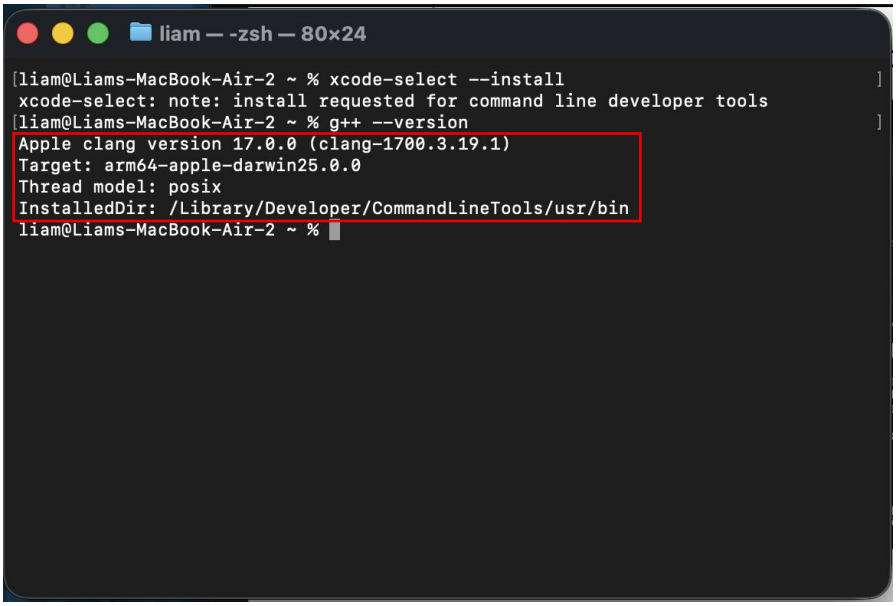
On macOS, open the ‘Terminal’ and enter `xcode-select --install`

Select the ‘Install’ button from the pop-up window.



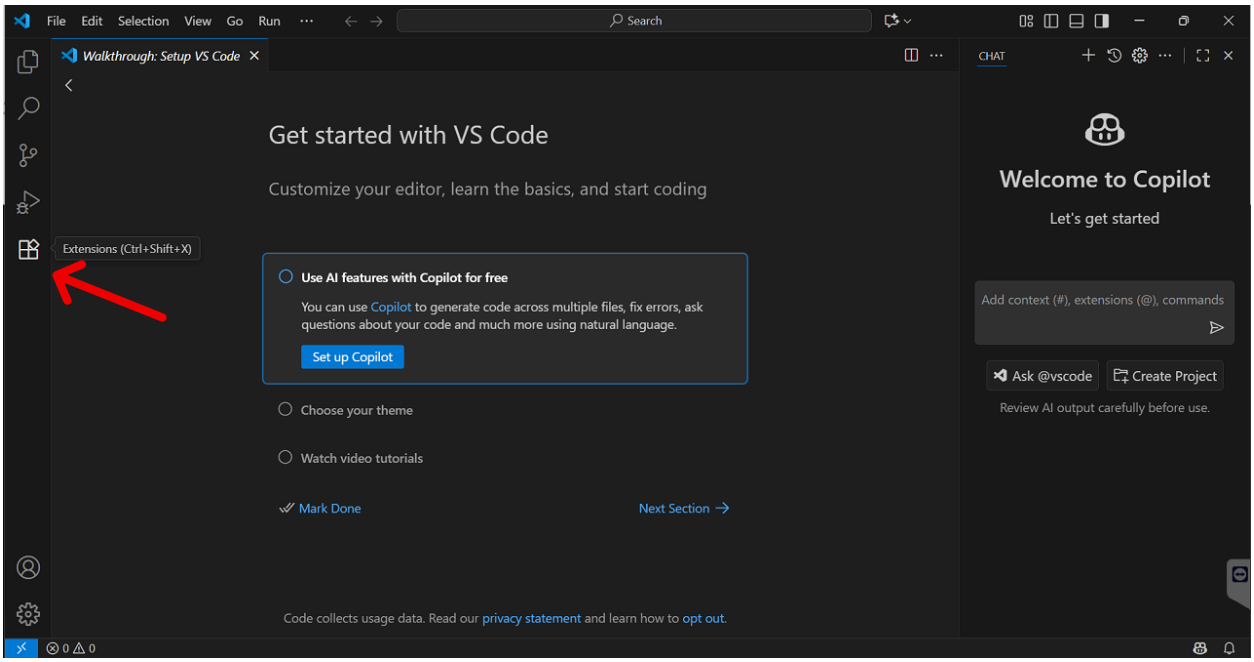
Once the installation is completed, in the terminal, verify your installation by entering `g++ --version`

You should see a similar output below:

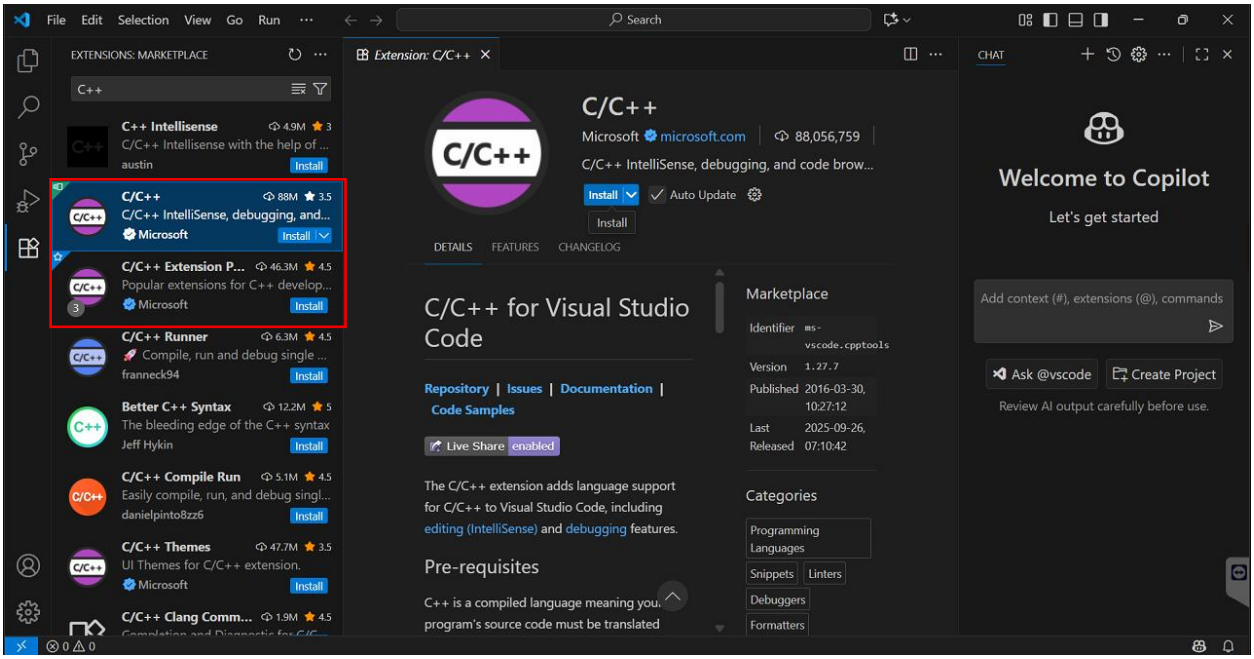


## 3. Installing Visual Studio Code Extensions (Windows & Mac)

Open Visual Studio Code and select the Extensions icon from the left toolbar.

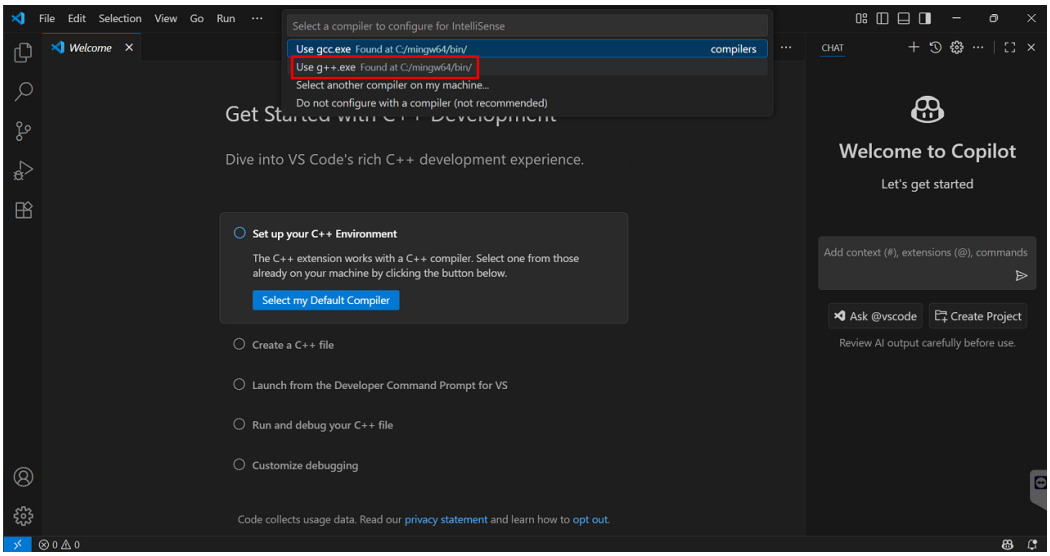



Enter "C++" into the search bar and install both "C/C++" and "C/C++ Extension Pack".

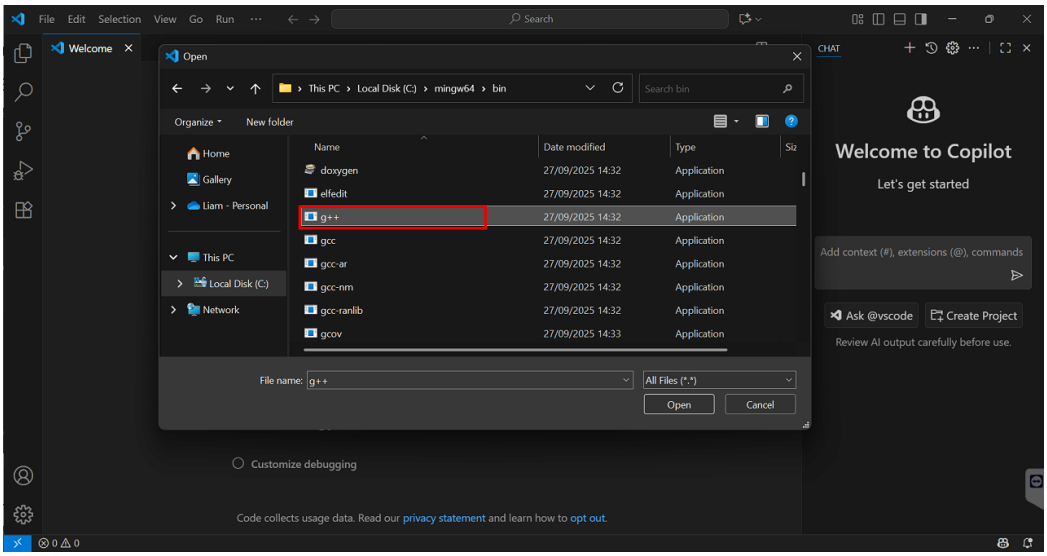


When the installation of the extensions is complete, the C++ setup screen will appear. At this point, you only need to select your default compiler.

Click on the button to open the drop-down menu and select:
**“Use g++.exe”** (or clang++ on Mac)




(Note: On Windows, a file explorer window may appear after selecting this option. Simply navigate to ‘C:/msys64/mingw64/bin’ and select ‘g++’.)




Visual Studio Code is now ready for C++ development.

# Section 2: Installing SFML 2.6.2

## 1. Downloading SFML 2.6.2

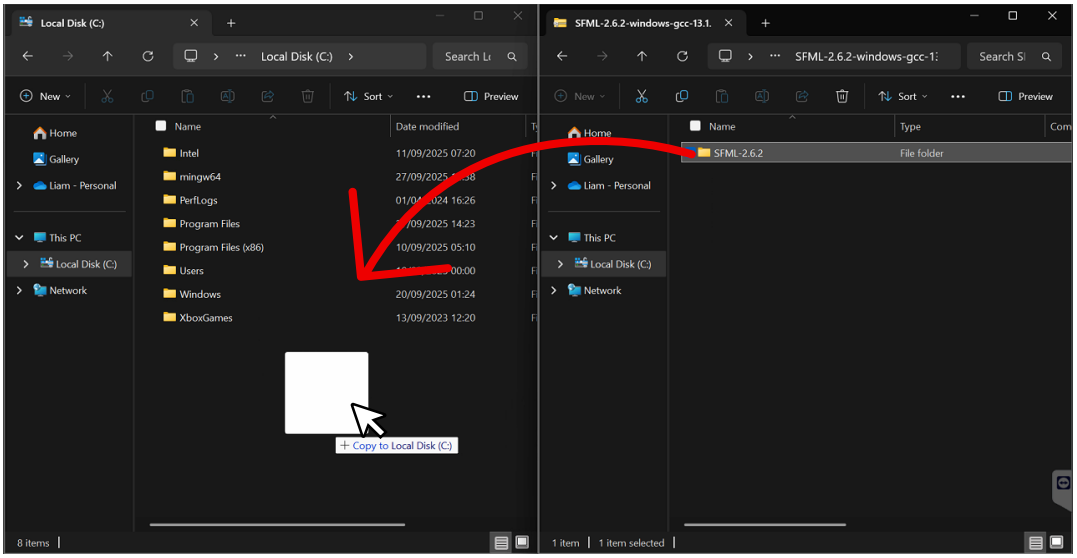
Follow the instructions for your respective operating system.

**(Mac Instructions begin on Page 11)**

## Windows:

Download and install the ‘GCC 13.1.0 MinGW (SEH) – 64-bit’ version of SFML 2.6.2 from https://www.sfml-dev.org/download/sfml/2.6.2/.


Inside the downloaded ZIP file, copy the ‘SFML-2.6.2’ folder to the root of your C: drive.

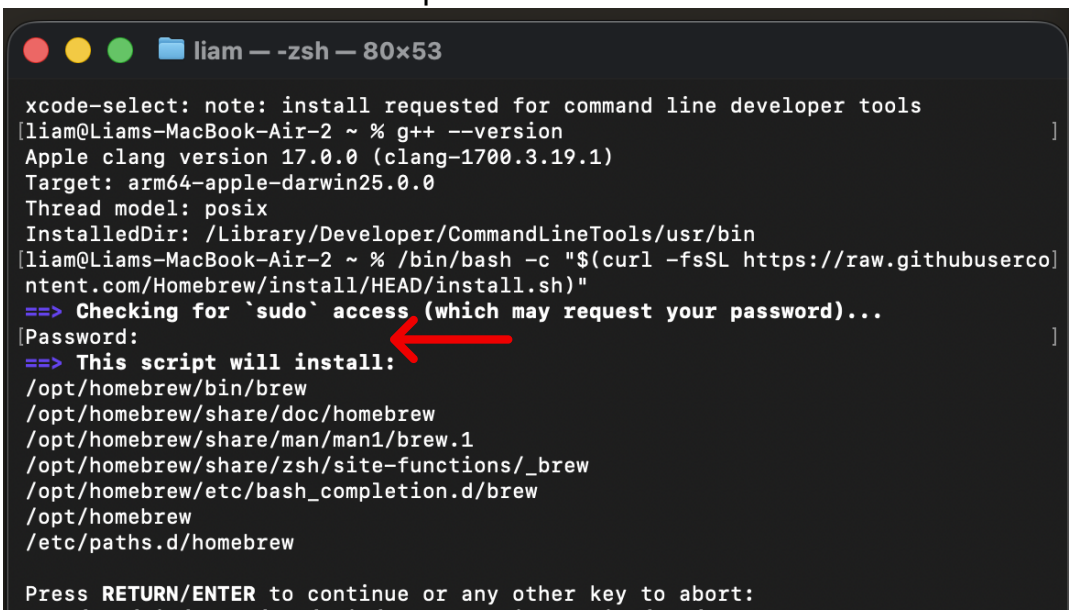


## Mac:

For simplicity, SFML will be installed using the Homebrew package manager. Go to https://brew.sh/ and select the copy button next to the installation command.


Paste and run the command in your terminal. You may be required to enter your password to continue. (When entering your password, characters will not show up, but this is normal. Type your password normally and press return/enter.)

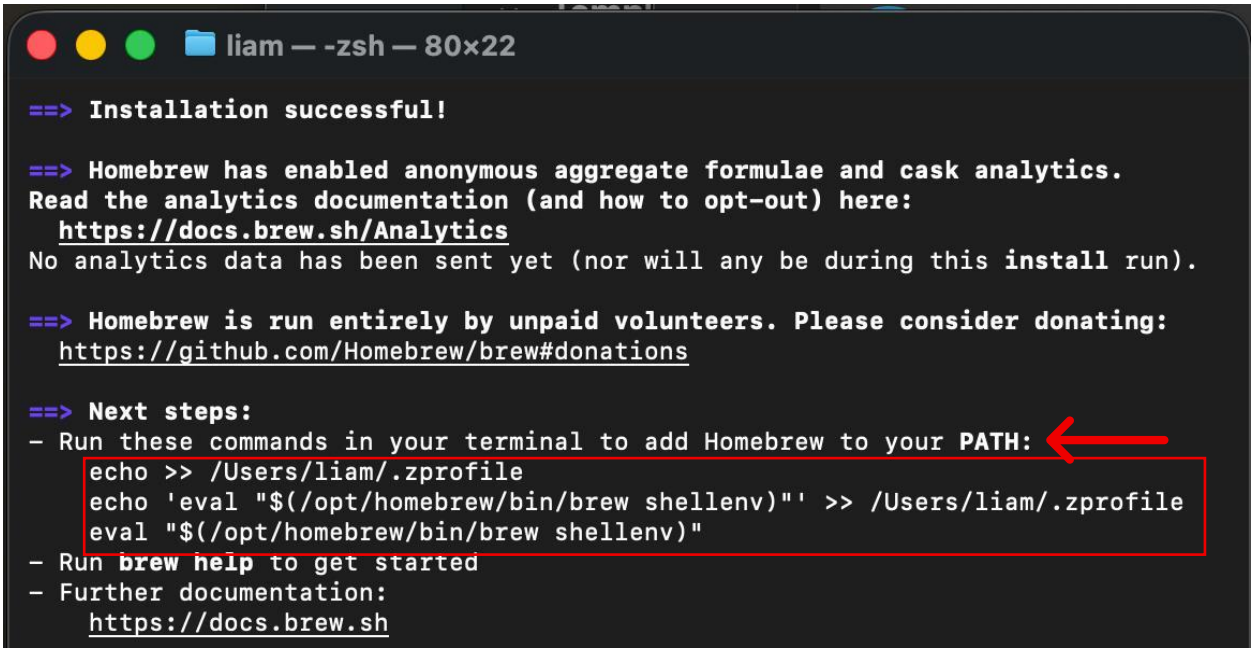
You should see a similar output below:




Once entered, press ‘Return / Enter’ to proceed with installation.

When the installation is complete, the installer will auto-generate the commands to add Homebrew to your PATH variable (as shown below).

Copy and paste these commands into your terminal and execute them before continuing. (They do not need to be copied individually.)

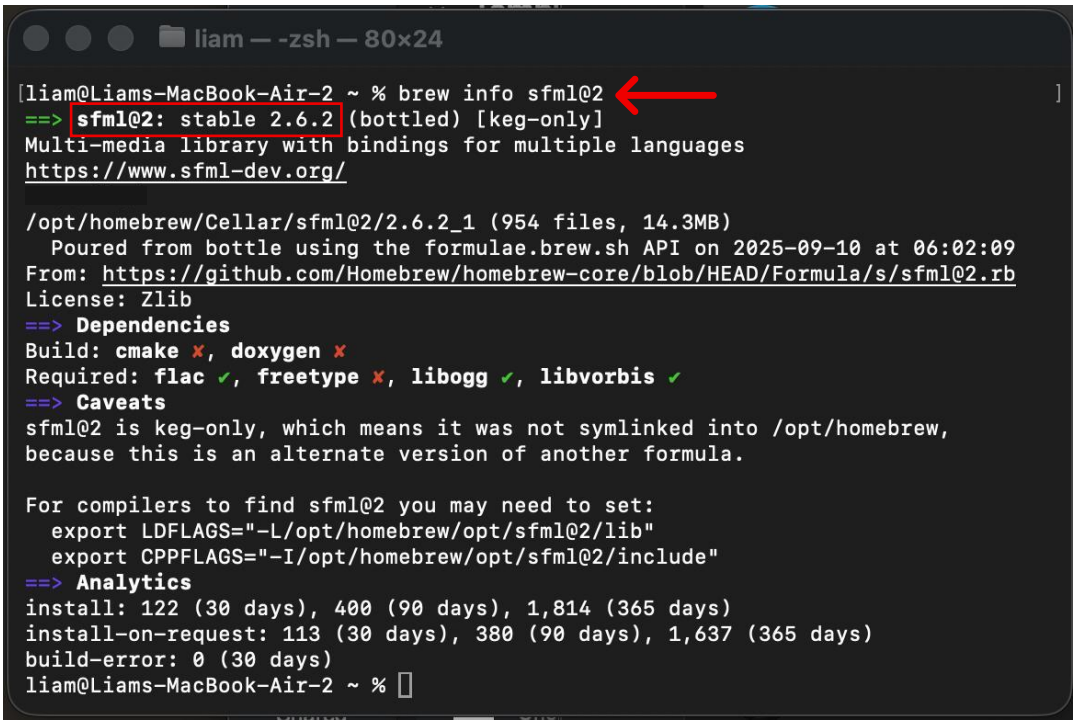



Once you have executed those commands, in the terminal, enter `brew --version` to verify your installation.

You should see a similar output below:

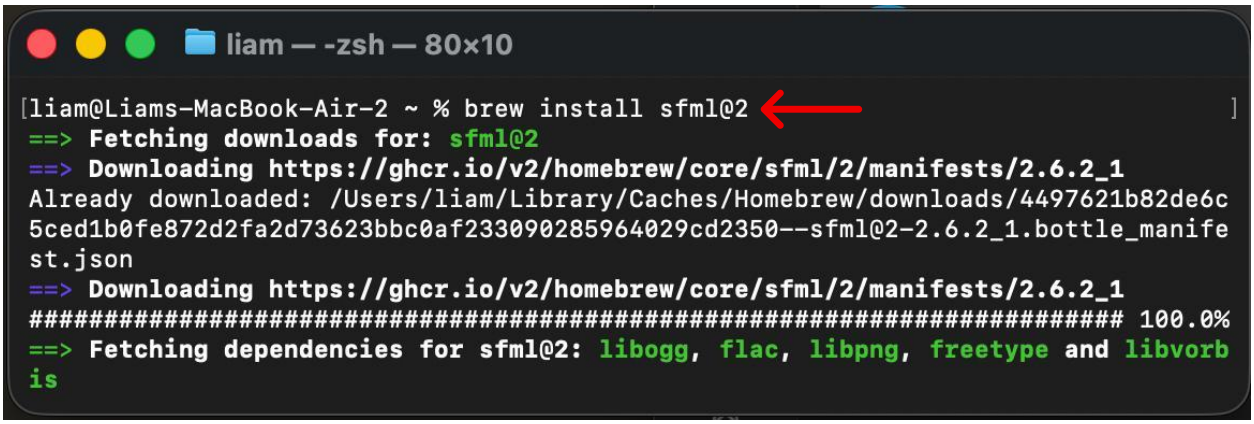
Then in your terminal, enter `brew info sfml@2` to verify that SFML 2.6.2 is available for download.

You should see a similar output below:

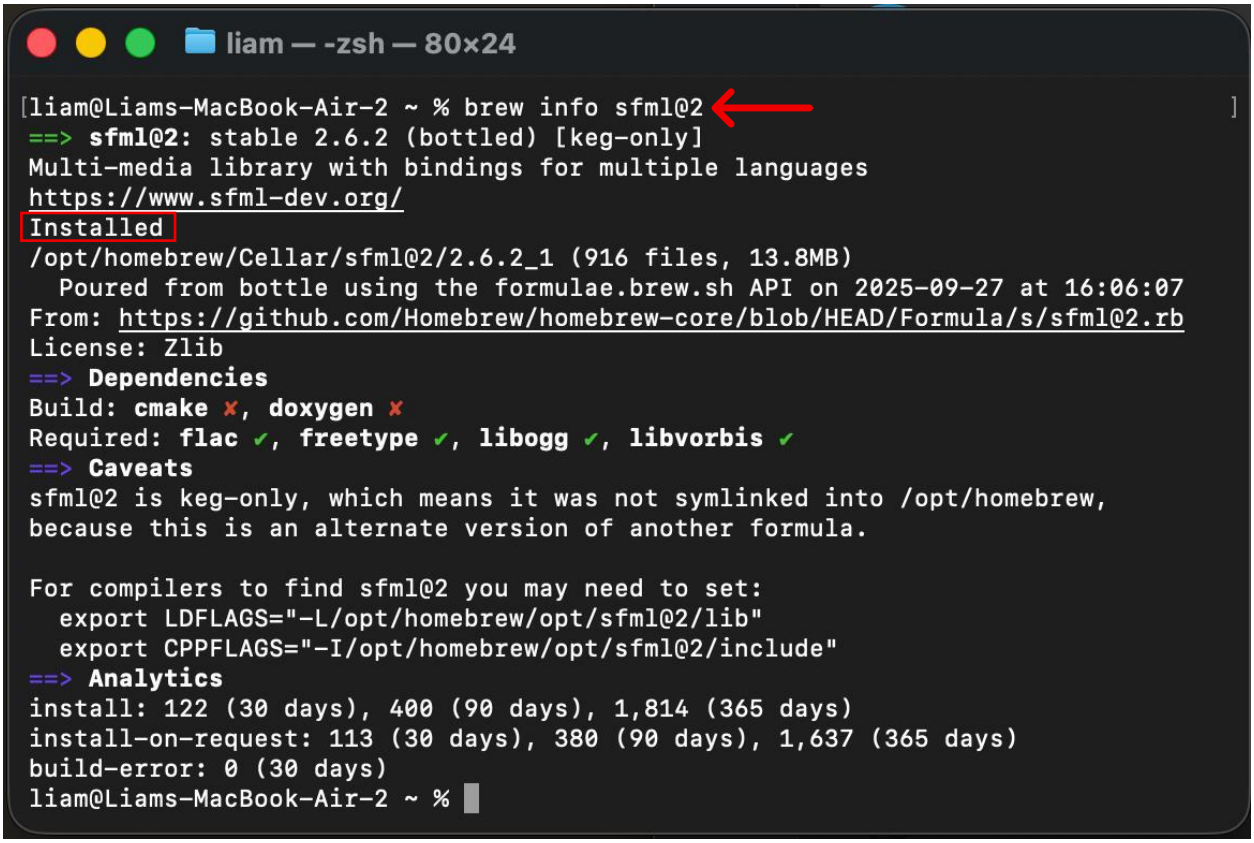



Now enter `brew install sfml@2` to download and install SFML.

You should see a similar output below:



Once complete, you can enter `brew info sfml@2` to verify that SFML 2.6.2 is installed correctly.




SFML has now been installed on your computer.

# Section 3: Configuring Visual Studio Code for SFML Projects

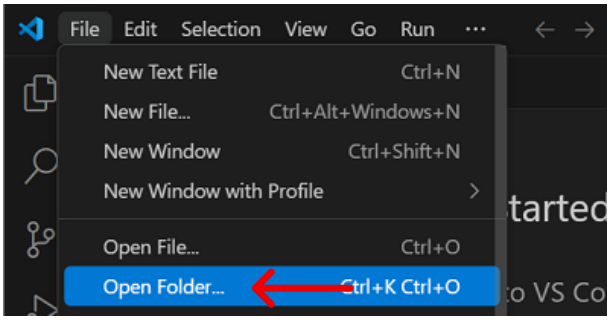
Now that Visual Studio Code is configured for C++ development, and SFML is installed on your computer, we must now tell Visual Studio Code how to build projects using the SFML libraries.

### 1. Building the Template Folder Structure

For the sake of this guide, we will create a template project folder that can be reused for new SFML projects.

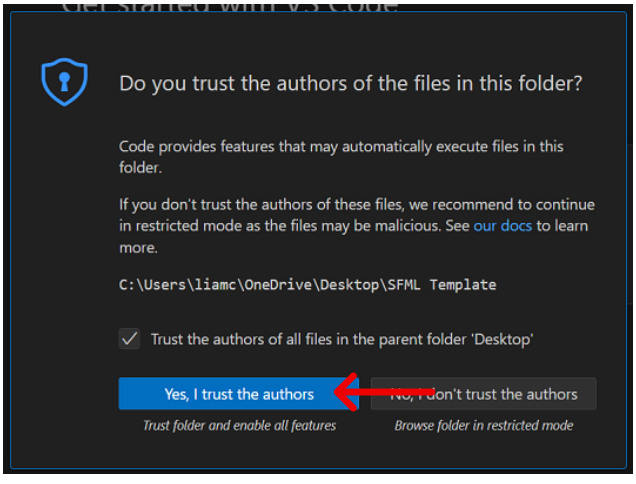
### 2. Creating Configuration Files

Open Visual Studio Code and select ‘File’ then ‘Open Folder’.



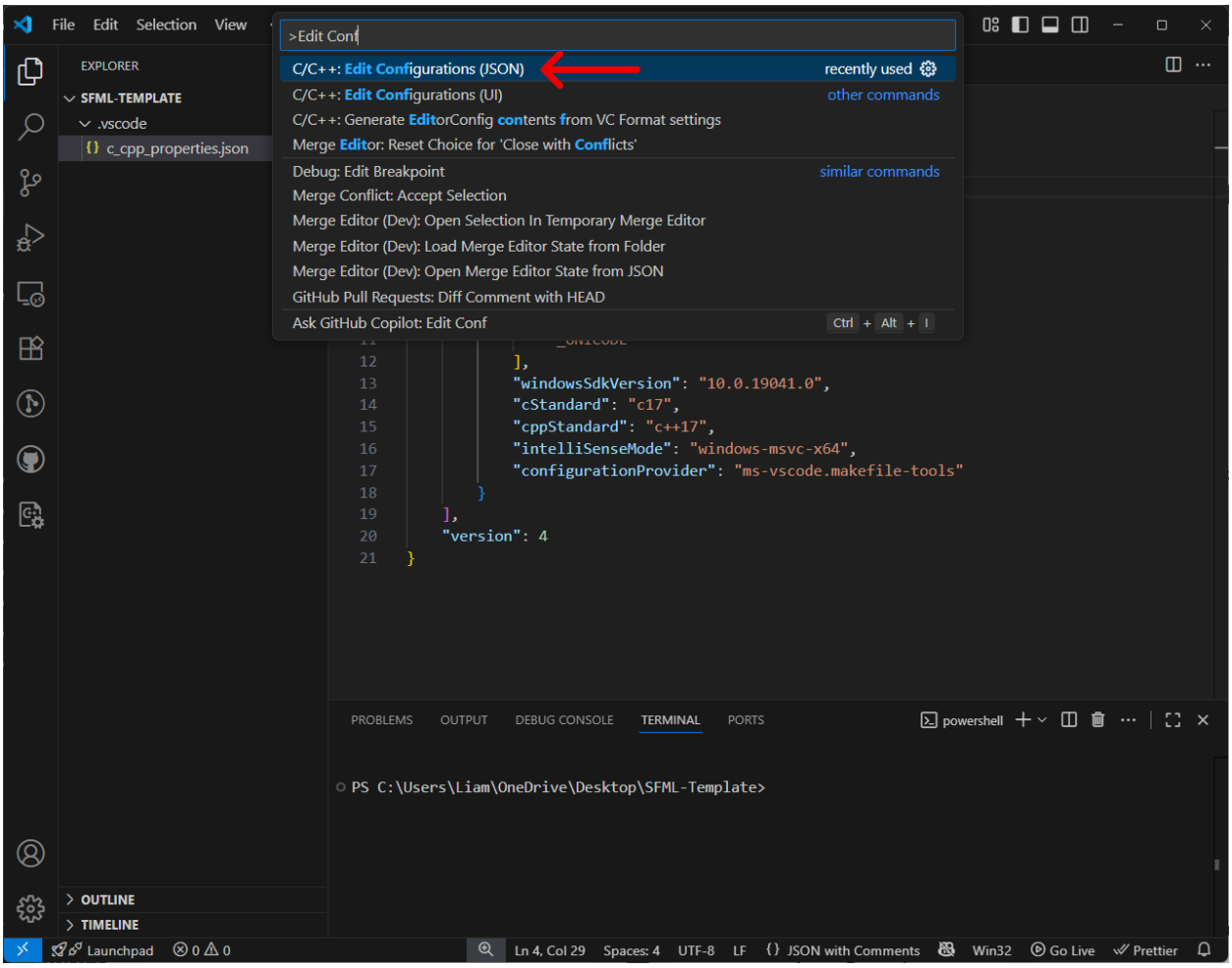

Then find and select the SFML Template folder you created.

Select “Yes, I trust the authors” if prompted.



Press ‘Ctrl + Shift + P’ (or Command + Shift + P on Mac) to open Visual Studio Code’s Command Palette.

In the text field, start typing:
`Edit Configurations` and select “C/C++: Edit Configurations (JSON).

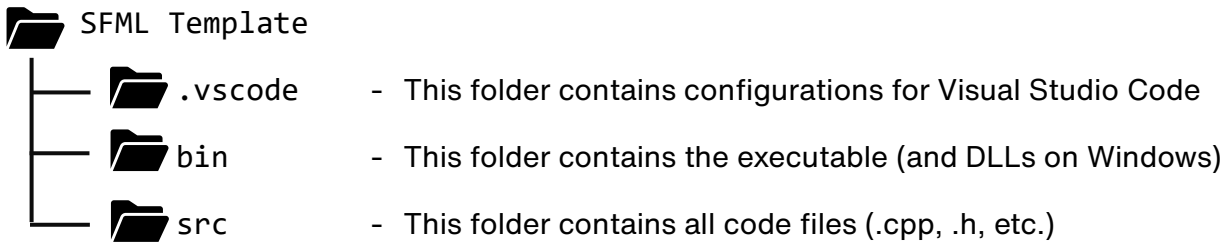



This will automatically create a ‘ .vscode’ folder with ‘c_cpp_properties.json” file (as shown above) inside of it. For now, save the file as is. We will modify it later.

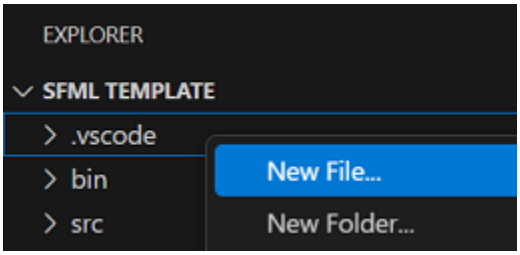
Now, in your ‘SFML Template’ folder, create two new folders, each named:

- bin
- src

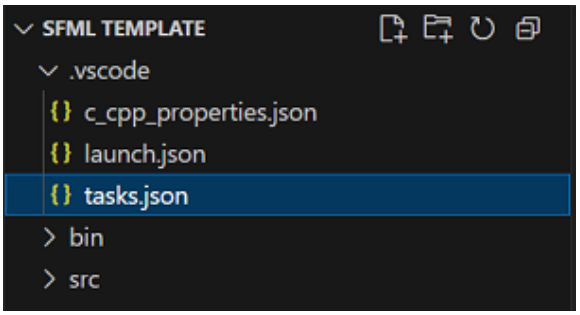
From the guide, your folder structure should look like this:



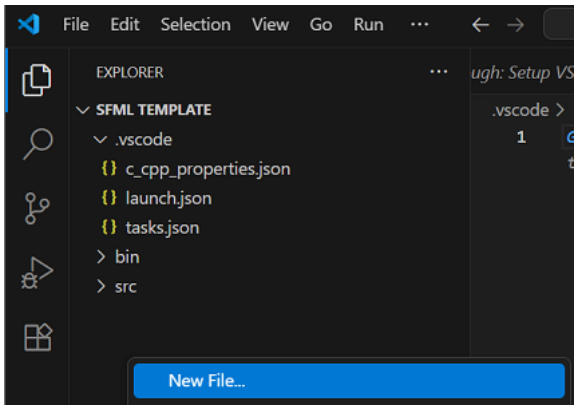

In Visual Studio Code, on the left, right-click on ‘ .vscode’ and select “New File”. Name this new file `c_cpp_properties.json`.



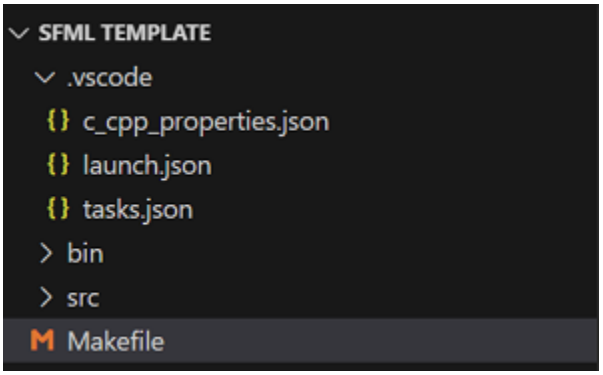

Repeat this process to also create two other files named: `launch.json` and `tasks.json`.



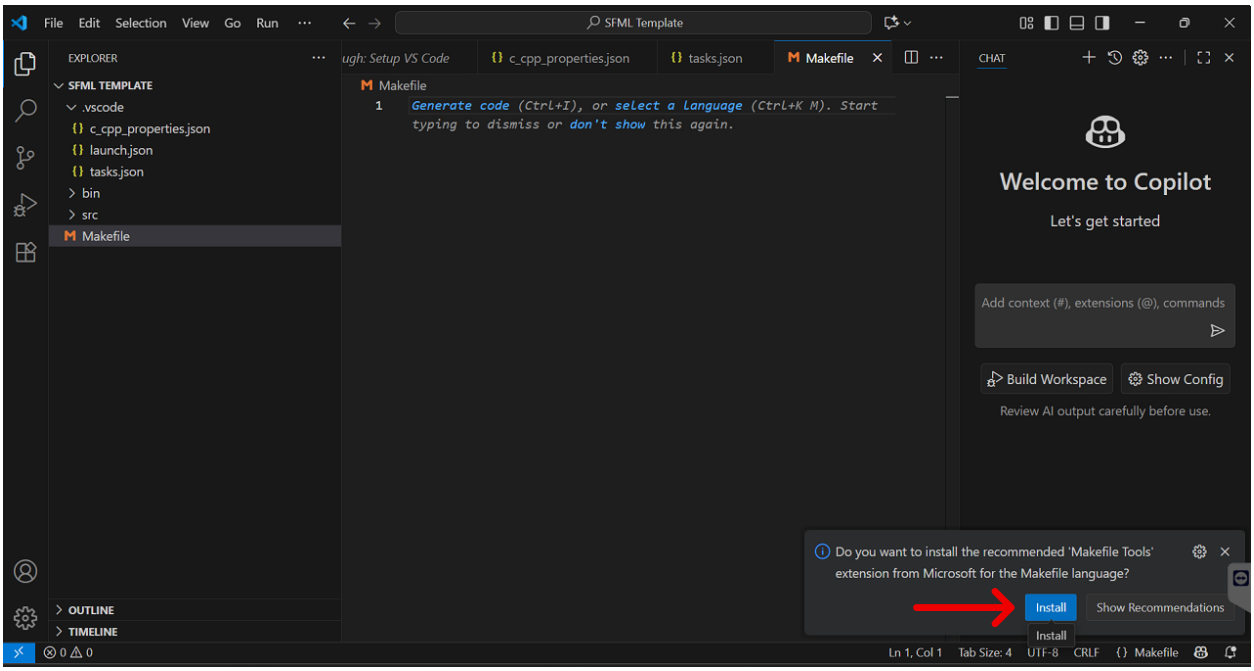

Then right-click in the empty space under ‘src’ and select “New File”.



Name this new file `Makefile`.




Also install the ‘Makefile Tools’ if prompted.



### 3. Populating Configuration Files

Follow the instructions for your respective operating system.

**(Mac Instructions begin on Page 21)**

## Windows:

Copy and paste the configurations for each file as shown below:

`c_cpp_properties.json`

```
{
  "configurations": [
    {
      "name": "windows-gcc-x64",
      "includePath": [
        "${workspaceFolder}/**",
        "C:/SFML-2.6.2/include" // SFML Installation Path (include)
      ],
      "compilerPath": "C:/msys64/mingw64/bin/g++.exe", // Compiler Installation Path (g++)
      "cStandard": "c17",
      "cppStandard": "c++17",
      "intelliSenseMode": "windows-gcc-x64",
      "defines": [],
      "compilerArgs": [
        "-Wall",
        "-Wextra"
      ]
    }
  ],
  "version": 4
}
```

`launch.json`

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Launch SFML App",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/bin/main.exe",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${workspaceFolder}/src",
      "externalConsole": true,
      "MIMode": "gdb",
      "miDebuggerPath": "C:/msys64/mingw64/bin/gdb.exe", // Compiler Installation Path (gdb)
      "preLaunchTask": "Build C++ program with SFML",
      "setupCommands": [
        {
          "description": "Enable pretty-printing for gdb",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ]
    }
  ]
}
```

`tasks.json`

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "Build C++ program with SFML",
            "type": "shell",
            "command": "g++",
            "args": [
                "-g",
                "src/*.cpp",
                "-I", "C:/SFML-2.6.2/include", // SFML Installation Path (include)
                "-L", "C:/SFML-2.6.2/lib", // SFML Installation Path (lib)
                "-lsfml-graphics",
                "-lsfml-window",
                "-lsfml-system",
                "-lsfml-audio",
                "-lsfml-network",
                "-std=c++17",
                "-o", "${workspaceFolder}/bin/main.exe"
            ],
            "options": {
                "cwd": "${workspaceFolder}",
                "shell": {
                    "executable": "C:/msys64/usr/bin/bash.exe",
                    "args": ["-c"]
                }
            },
            "problemMatcher": ["$gcc"],
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "detail": "Build all cpp files for SFML"
        },
    ]
}
```

`Makefile`

```
# Path to SFML installed
SFML_PATH = C:/SFML-2.6.2

# Automatically find all .cpp files in src/
CPPFILES := $(shell find ./src -type f -name "*.cpp")

# Compiler and flags
CXX = g++
CXXFLAGS = -std=c++17 -g -I$(SFML_PATH)/include
LDFLAGS = -L$(SFML_PATH)/lib -lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio -lsfml-
network

# Output binary
OUTPUT = bin/main.exe

# Default target
all: $(OUTPUT)

$(OUTPUT): $(CPPFILES)
	mkdir -p bin
	$(CXX) $(CPPFILES) $(CXXFLAGS) -o $(OUTPUT) $(LDFLAGS)

clean:
	rm -f $(OUTPUT)
```

(Note: If you have installed your compiler or SFML library in a location that is different from which is specified in this guide, please alter the SFML Paths and Compiler Paths as highlighted above.) **Then, ensure you save each file.**

## Mac (Apple Silicon):

Copy and paste the configurations for each file as shown below:

`c_cpp_properties.json`

```
{
    "configurations": [
        {
            "name": "Mac",
            "includePath": [
                "${workspaceFolder}/**",
                "/usr/local/Cellar/sfml@2/2.6.2_1/include",
                "/opt/homebrew/Cellar/sfml@2/2.6.2_1/include"
            ],
            "defines": [],
            "cStandard": "c17",
            "cppStandard": "c++14",
            "compilerPath": "/usr/bin/clang++",
            "intelliSenseMode": "macos-clang-arm64",
            "macFrameworkPath": [
                "/System/Library/Frameworks",
                "/Library/Frameworks"
            ]
        }
    ],
    "version": 4
}
```

`launch.json`

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Launch SFML App",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/bin/main",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${workspaceFolder}/src",
      "environment": [],
      "externalConsole": false,
      "MIMode": "lldb",
      "targetArchitecture": "arm64",
      "preLaunchTask": "Build SFML Project",
      "setupCommands": [
        {
          "description": "Enable pretty-printing for gdb/lldb",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ]
    }
  ]
}
```

... continued on next page ...

`tasks.json`

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Build SFML Project",
      "type": "shell",
      "command": "make",
      "options": {
        "cwd": "${workspaceFolder}"
      },
      "group": {
        "kind": "build",
        "isDefault": true
      }
    }
  ]
}
```

`Makefile`

```
# Detect Homebrew prefix dynamically
BREW_PREFIX := $(shell brew --prefix)

# Path to SFML installed via Homebrew
SFML_PATH = $(BREW_PREFIX)/Cellar/sfml@2/2.6.2_1

# Automatically find all .cpp files in src/
cppFileNames := $(shell find ./src -type f -name "*.cpp")

# Compiler and flags
CXX = clang++
CXXFLAGS = -std=c++14 -g -I$(SFML_PATH)/include
LDFLAGS = -L$(SFML_PATH)/lib -lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio -lsfml-
network

# Default target
all: compile

compile:
	mkdir -p bin
	$(CXX) $(cppFileNames) $(CXXFLAGS) -o bin/main $(LDFLAGS)

clean:
	rm -rf bin/main
```

**Then, ensure you save each file.**

# Section 4: Running Your First SFML Program

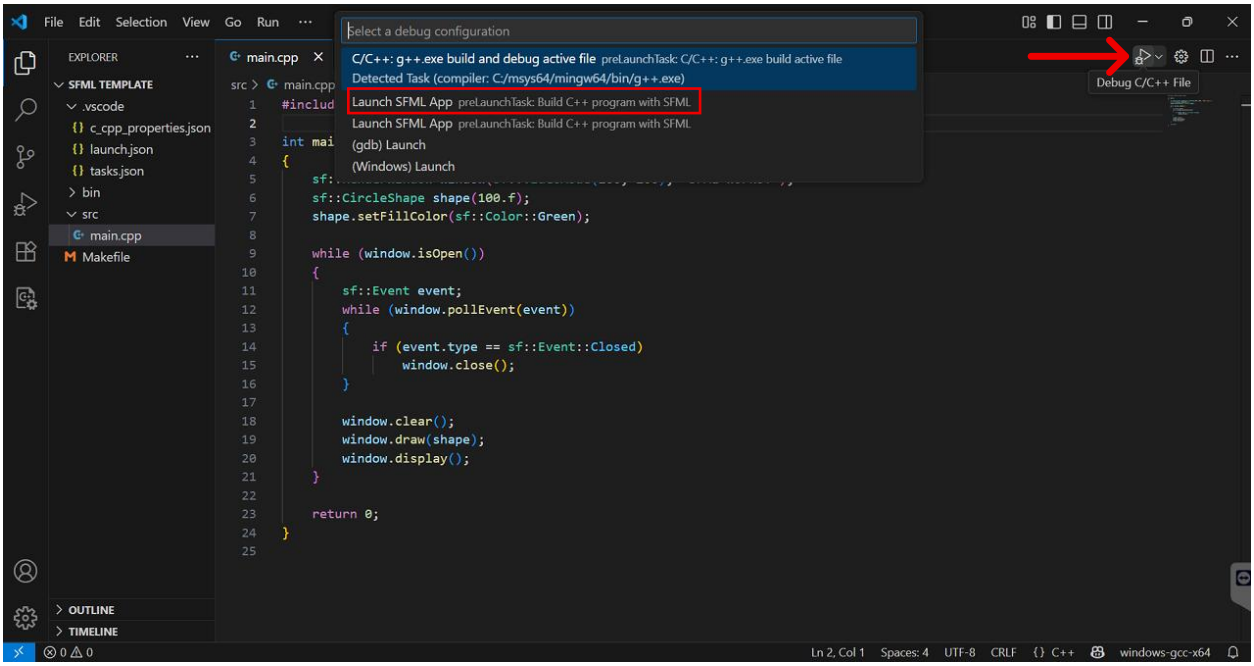
In your ‘src’ folder, create a file called `main.cpp` and paste the following sample SFML code into that file.

`main.cpp`

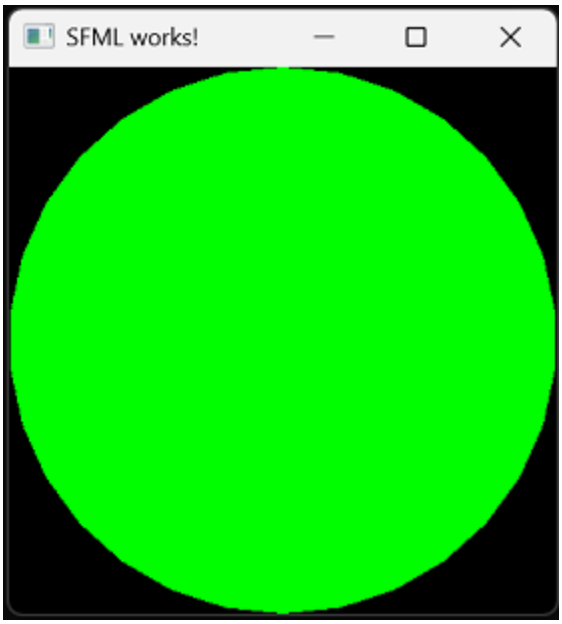
```
#include <SFML/Graphics.hpp>

int main()
{
    sf::RenderWindow window(sf::VideoMode(200, 200), "SFML works!");
    sf::CircleShape shape(100.f);
    shape.setFillColor(sf::Color::Green);

    while (window.isOpen())
    {
        sf::Event event;
        while (window.pollEvent(event))
        {
            if (event.type == sf::Event::Closed)
                window.close();
        }

        window.clear();
        window.draw(shape);
        window.display();
    }

    return 0;
}
```

Then, select the ‘Run/Debug’ icon on the top-right of your window.

When the drop down menu appears, select `Launch SFML App`.



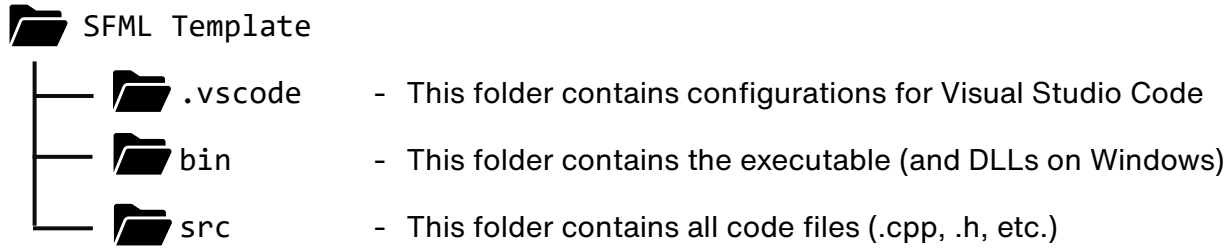
Your C++ code will now be compiled and your SFML window will appear!



# Section 5: Setting Up SFML Projects for GitHub Collaboration

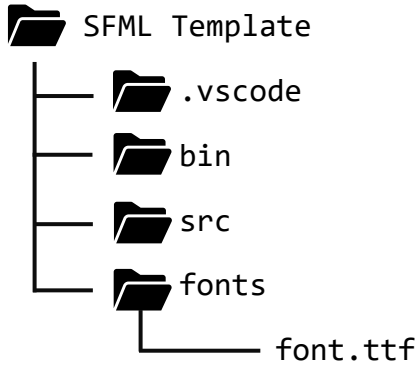
### 1. Understanding Your Project Structure

From the guide, your folder structure should look like this:




When adding additional assets to your projects, such as fonts, images, etc., these should be placed either in the ‘SFML Template’ folder or preferably in a dedicated folder.

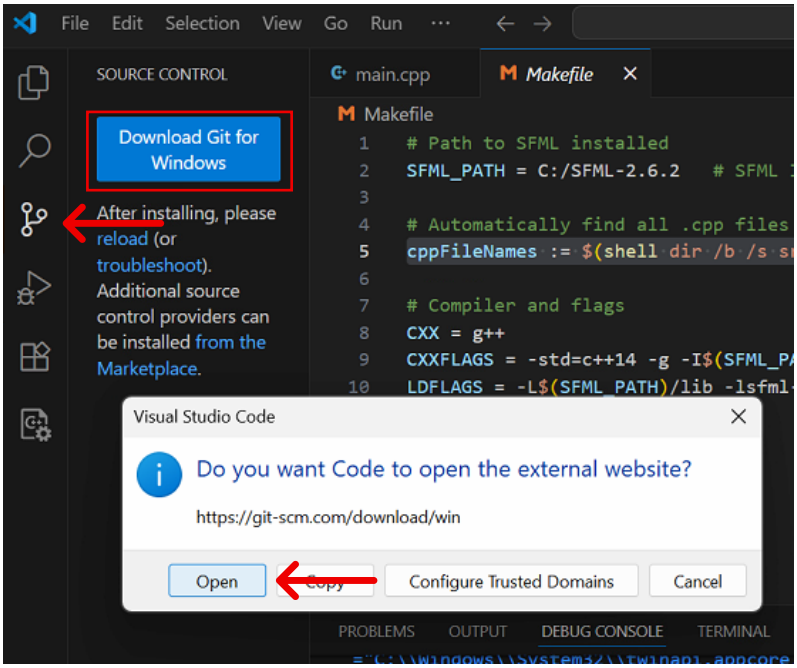
For example, if you were to include a font file with your project, your folder structure should look like this:



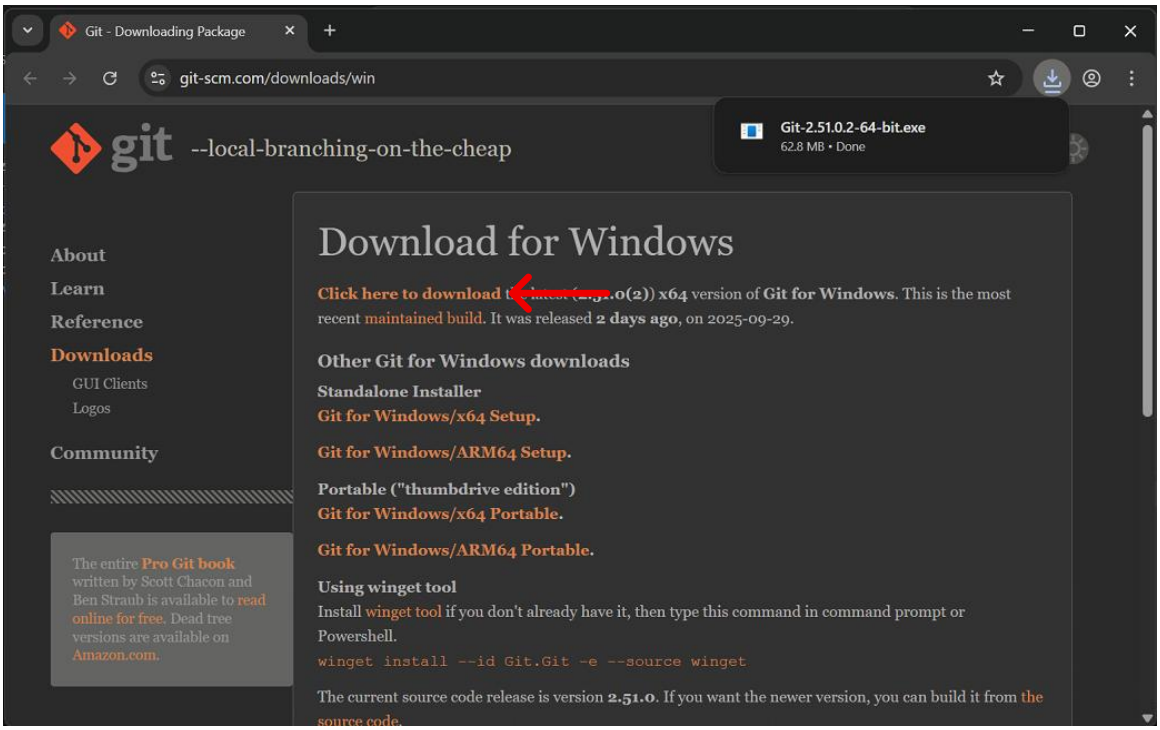

This is done to allow Visual Studio Code to correctly find your additional files when compiling your project.

## 2. Installing Git on Visual Studio Code (Windows ONLY)

To use GitHub source control integrated into Visual Studio Code, select the ‘Source Control’ icon on the left toolbar and select ‘Download Git for Windows’.




Download and install the latest version of Git from the website using the default settings in the installer. Then close and reopen Visual Studio Code.

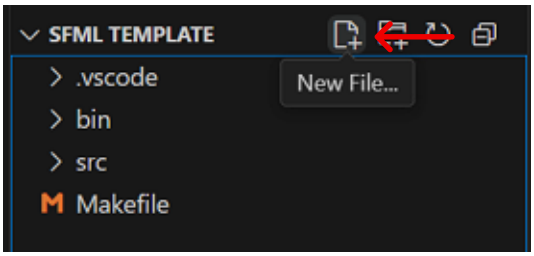


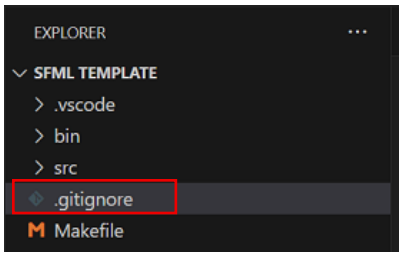
### 3. Setting Up Your Folder for GitHub Collaboration

The key to creating a GitHub repository that works for all members of a group without being affected by system specific files (such as those we setup earlier) is to create a `.gitignore` file.

This file will only upload the files necessary for members of your project to work on, without affecting your local Visual Studio project setup.

To create this file, click on the 'New File' button next to 'SFML Template' in your 'Explorer' side menu and name the file `.gitignore`.




Then enter the following text into your newly created file and save it:

`.gitignore`

```
/*

// Folders to include
!/src/

// Windows
*.dll
*.exe

// Mac
.DS_Store
**/.DS_Store
```

- This line indicates to ignore all files
- These lines indicate to make an exception for the /src folder (as this code contains your project code) and the .gitignore file.
- Additional lines are included to ignore operating system specific files.

In the example discussed above, if there was another folder present, such as one for fonts or other assets, those should be included in this file as well, such as shown below.

`.gitignore`

```
…
!/src/
!/fonts/
…
```

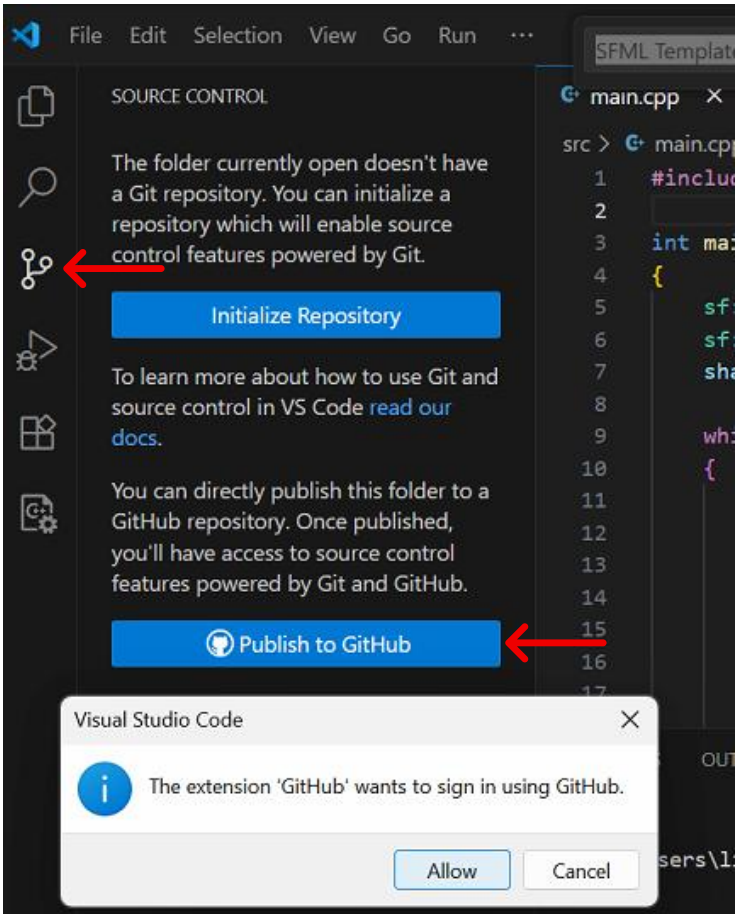
### 4. Publishing Your GitHub Repository

If you haven't already, you need to configure your initial Git username and email.

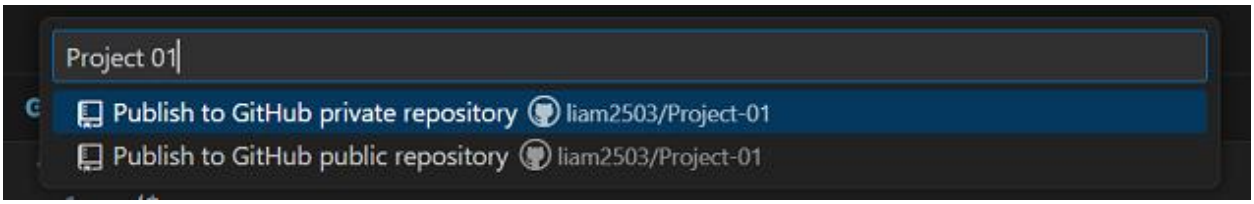
To do this, open the terminal in Visual Studio Code and enter the following commands (replacing the respective fields with your account information).

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

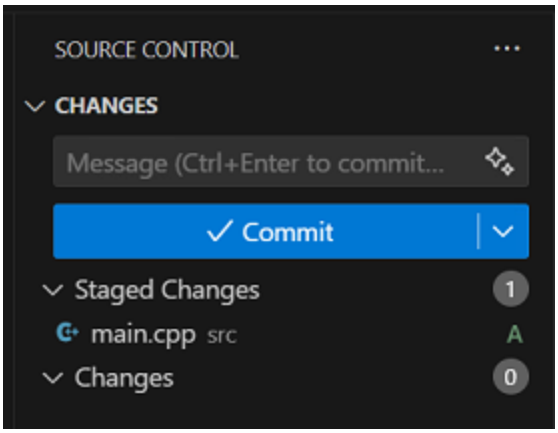
Once completed, elect the 'Source Control' icon from the left toolbar and then select 'Publish to GitHub'. Sign In to GitHub when prompted or create an account if you haven't already.



Then select what you would like to name your repository and whether you would like to publish to a ‘Public’ or ‘Private’ repository.

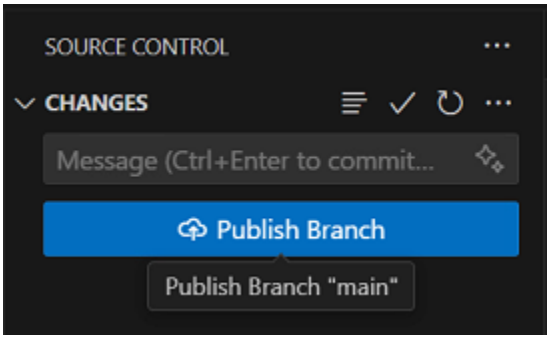


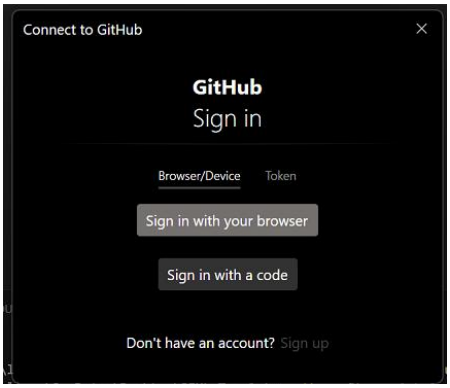

You should then see a summary of all files that will be uploaded to your repository.




Type a brief description of your changes to your code into the ‘message’ field and then select ‘Commit’.

Then select “Publish Branch” to synchronize your project with the GitHub repository. (You may be prompted to Sign In again, but this is only a one-time process).



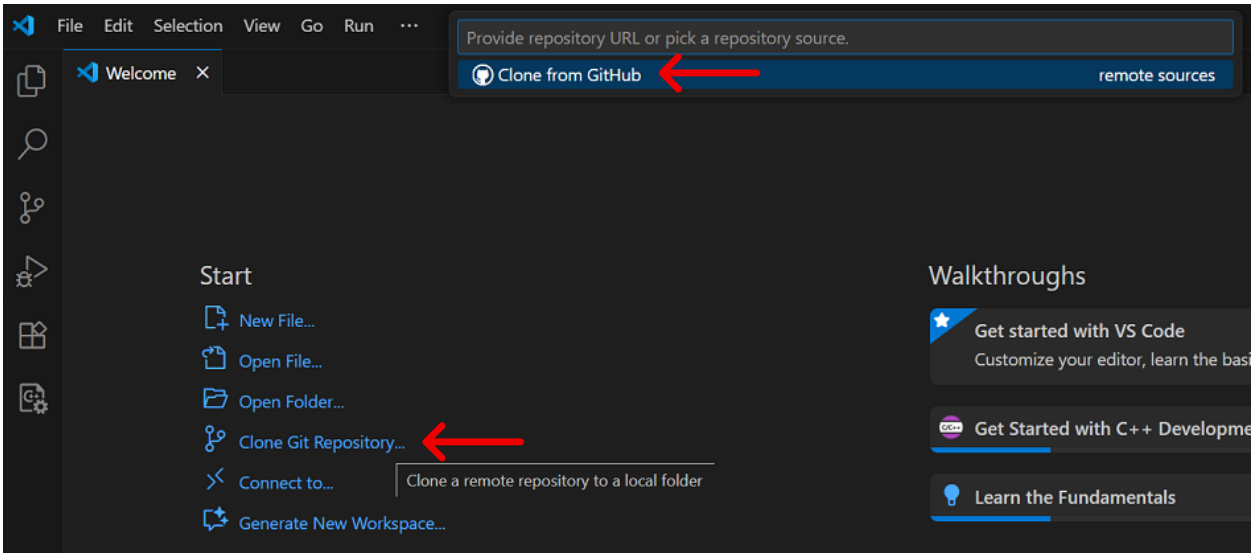




You have now successfully published your SFML project to GitHub.

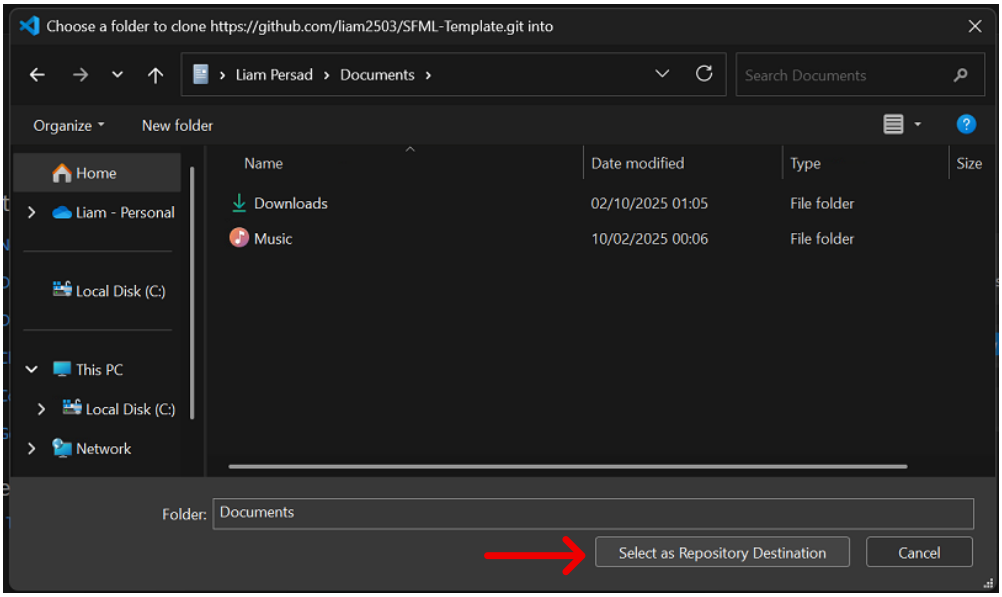
## 5. Cloning From an Existing Repository

If you are cloning another repository setup by someone else using this guide, follow these steps.

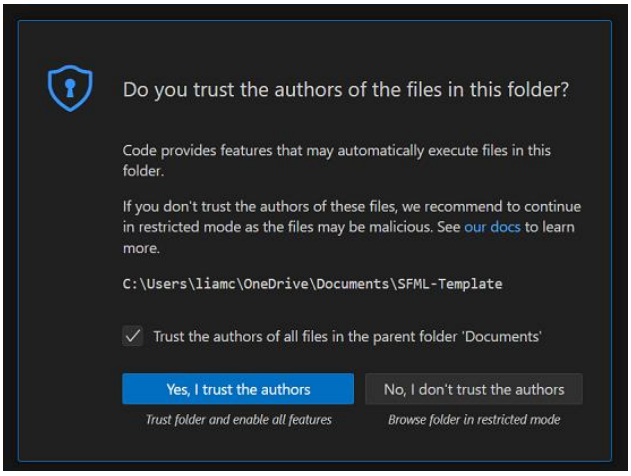
On the Visual Studio Code ‘Welcome Screen’, select ‘Clone Git Repository...’ and then select ‘Clone from GitHub’.



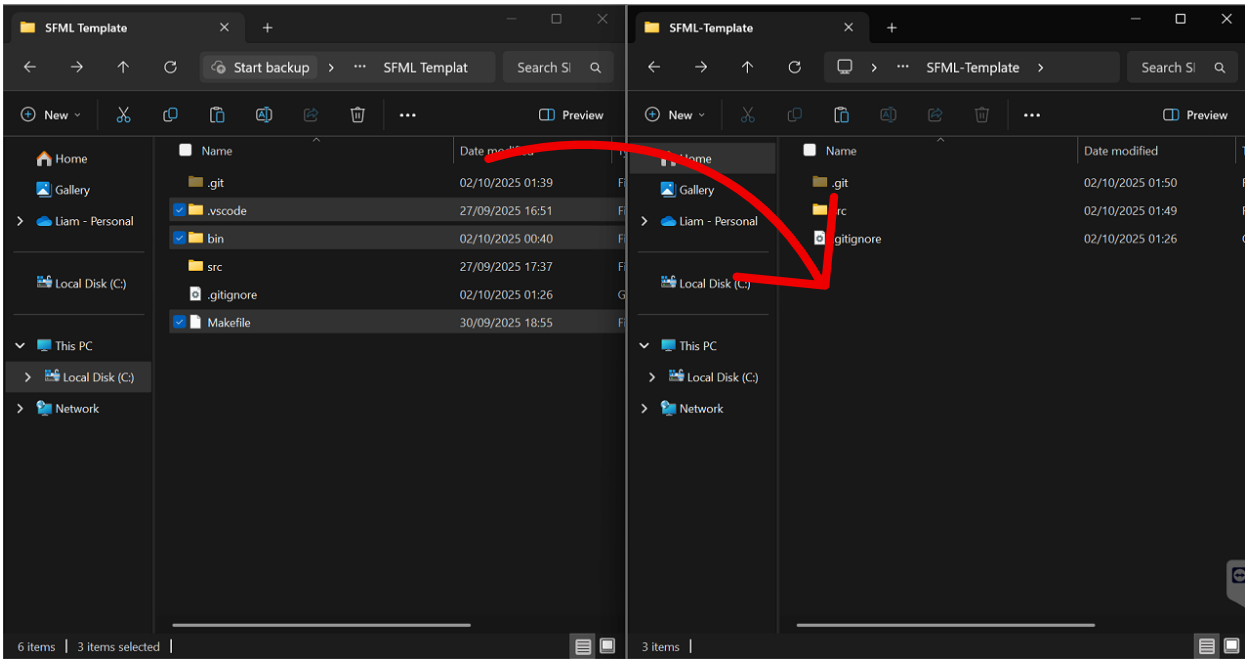

A list of available repositories will appear. Select the repository you want to clone and choose a location on your computer to save it.
(Note: If you have not already been invited to and accepted your invitation to collaborate on a repository, it will not show up here. You must do this before trying to clone.)



When prompted, open the repository and select ‘Yes, I trust the authors’.




Then, from the ‘SFML Template’ folder previously created, copy (.vscode, bin and Makefile) into the cloned repository folder.
(On Mac, the .vscode folder may be hidden. Press “Command” + “Shift” + “ . ” to show hidden folders.)




Your repo is now ready to be run in Visual Studio Code.

(In rare cases, the name of your project folder may cause issues when compiling your code. Simply close Visual Studio Code, rename your folder and then reopen Visual Studio code and try compiling again.)

## References / Credits:


- YouTube. *Installing SFML on Mac*. Available at: https://www.youtube.com/watch?v=4UaqmTvq2zk
- SFML-Dev.org. *Getting Started with Code::Blocks*. Available at: https://www.sfml-dev.org/tutorials/2.6/start-cb.php
- Beatzoid. *sfml-macos* [GitHub repository]. Available at: https://github.com/Beatzoid/sfml-macos
- Stack Overflow. *How do I use SFML on Apple M1 Mac?* Available at: https://stackoverflow.com/questions/69720807/how-do-i-use-sfml-on-apple-m1-mac/73402250#73402250